МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева-КАИ» (КНИТУ-КАИ)
Институт Компьютерных технологий и защиты информации

Кафедра Систем информационной безопасности

ОТЧЁТ

по лабораторной работе №2
по дисциплине
«Аппаратные средства телекоммуникационных систем»
на тему:

«РАСПАРАЛЛЕЛИВАНИЕ РЕСУРСОЁМКИХ ЦИКЛОВ: ПЕРЕМНОЖЕНИЕ МАТРИЦ, ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ»

Выполнил:
Студент гр. 4404
Устеников К. А.

Казань, 2026

Цель работы:

Освоить распараллеливание вычислительно тяжёлой задачи (перемножение квадратных матриц) на архитектуре с разделяемой памятью, научиться избегать состояний гонки за счёт разделения данных по потокам, применять модель fork-join и оценивать эффективность параллелизма через коэффициент ускорения. Выполнена задача 4 пособия «Параллельное программирование на OpenMP».

Язык реализации C#, в котором директивы OpenMP отсутствуют, поэтому конструкции пособия воспроизводятся штатными средствами языка. Соответствие для данной работы:

Конструкция OpenMP	Средство C# в работе
#pragma omp parallel for schedule(static) по строкам	массив Thread[], каждому потоку назначается непрерывный диапазон строк C, запуск Start()
неявный барьер параллельной области	цикл Join() (главный поток ждёт всех, затем сравнивает результаты)
отсутствие разделяемой записи (нет critical/atomic)	непересекающиеся диапазоны записи в C; A,B доступны только на чтение
проверка корректности (reduction по сумме)	суммирование выполняется в главном потоке **после** барьера, поэтому синхронизация при редукции не требуется
OMP_NUM_THREADS	аргумент командной строки <threads>

Задание:

Задача 4. Реализовать последовательную и параллельную версии перемножения двух квадратных матриц размера N×N (N = 1000). Параллелирование выполнить по строкам матрицы результата. Сравнить время выполнения при 2, 4 и 8 потоках, вычислить коэффициент ускорения и проверить совпадение результатов обеих версий по контрольной сумме. Ответить на вопросы: почему в задаче не требуются мьютексы; как выбран порядок обхода матриц и зачем; почему ускорение меньше линейного.

Общие требования. Число потоков и размер матрицы подаются через командную строку. Использование библиотечных средств, выполняющих перемножение целиком (например, встроенных BLAS-обёрток), запрещено —

арифметика пишется вручную. Каждый запуск сопровождается скриншотом с видимой командой и полным выводом.

Ход работы:

Матрицы хранятся одномерными массивами double длины N*N, элемент (i,j) адресуется как m[i*N+j] — это обеспечивает последовательный доступ к памяти и эффективное использование кэша. Вычисление элемента результата — скалярное произведение строки A и столбца B; чтобы оба операнда читались подряд по памяти, выбран порядок вложенных циклов i → k → j: для фиксированных i,k внутренний цикл по j проходит по строке B[k] последовательно, а коэффициент A[i*N+k] выносится в локальную переменную aik и переиспользуется. При альтернативном порядке i → j → k матрица B читалась бы по столбцам с шагом N, что резко увеличивает число промахов кэша.

Параллелирование выполнено по строкам матрицы C: при T потоках каждый получает непрерывный блок строк длиной N/T (последний поток добирает остаток до N). Это прямой аналог parallel for co schedule(static). Поскольку потоки пишут в непересекающиеся строки C, а A и B не изменяются, разделяемая запись отсутствует — состояние гонки невозможно по построению, поэтому блокировки (lock) и атомарные операции в вычислительном ядре не применяются. Более того, разбиение по строкам не изменяет порядок арифметических операций внутри каждой строки, поэтому параллельная версия воспроизводит последовательную побитово точно; совпадение подтверждается сравнением контрольных сумм после барьера Join().

Фрагмент вычислительного ядра (lab2/Program.cs):

```
static void Mul(double[] A, double[] B, double[] C, int N, int r0, int r1)
{
    for (int i = r0; i < r1; i++)
        for (int k = 0; k < N; k++)
        {
            double aik = A[i * N + k];
```

```
            for (int j = 0; j < N; j++)
                C[i * N + j] += aik * B[k * N + j];
        }
}
```

Фрагмент распараллеливания по строкам с барьером:

```
int rows = N / T;
for (int t = 0; t < T; t++)
{
    int r0 = t * rows, r1 = (t == T - 1) ? N : r0 + rows;
    th[t] = new Thread(() => Mul(A, B, Cp, N, r0, r1));
    th[t].Start();
}
foreach (var x in th) x.Join();   // неявный барьер: ждём всю команду потоков
```

Команды запуска (три повтора одной команды):

```
cd lab2
dotnet run -- 2 1000
dotnet run -- 4 1000
dotnet run -- 8 1000
```

На рисунке 1 представлен запуск с 2 потоками: видны время последовательной и параллельной версий, коэффициент ускорения и совпадение контрольных сумм (Results Match: True).

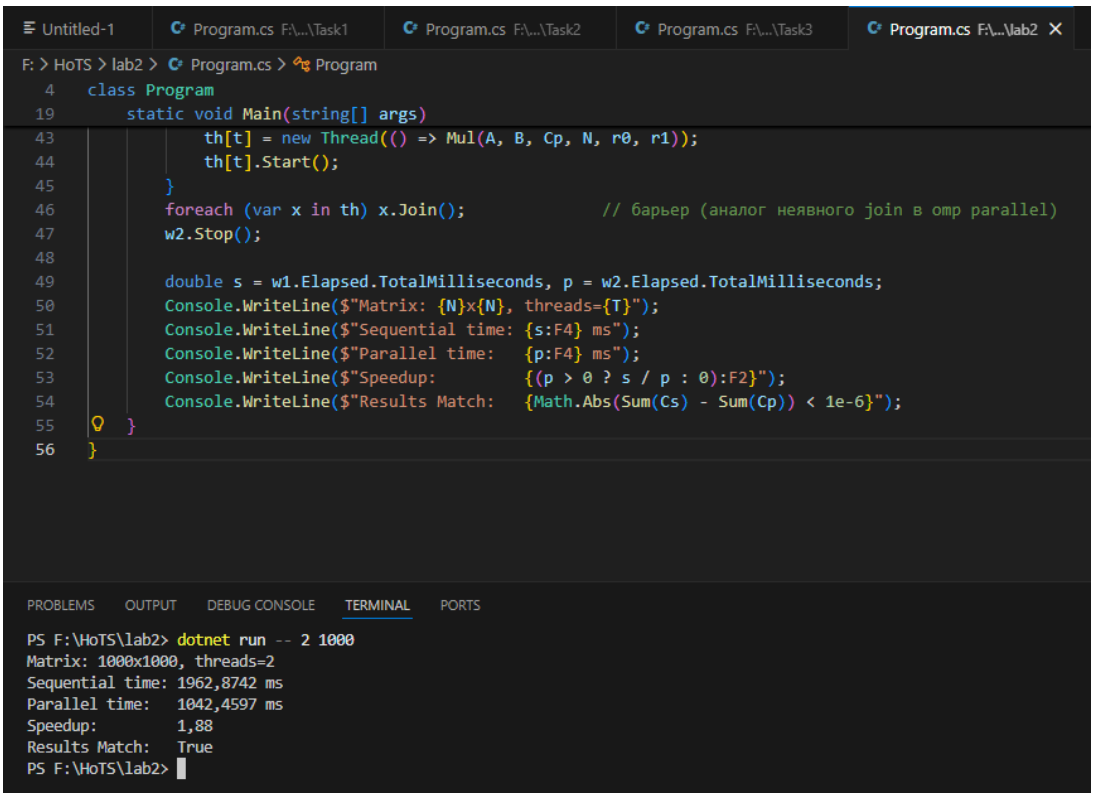


Рисунок 1. Перемножение матриц 1000×1000, 2 потока

На рисунке 2 запуск с 4 потоками. Параллельное время уменьшается, ускорение возрастает относительно рисунка 1; результат по-прежнему совпадает с эталоном.

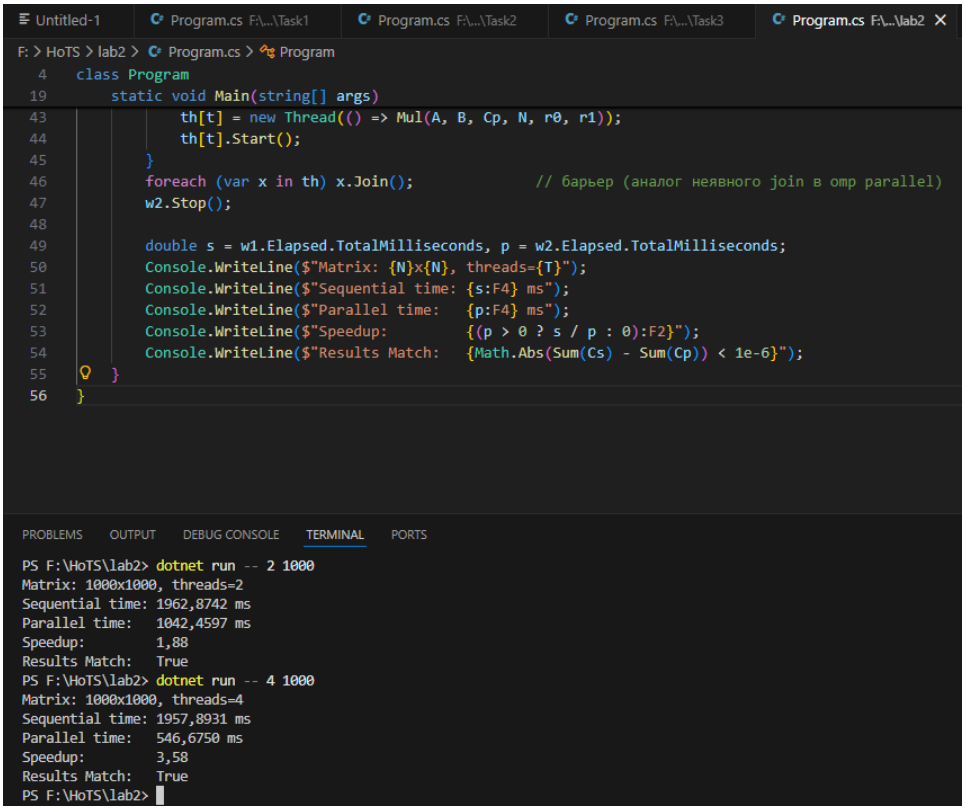


Рисунок 2. Перемножение матриц 1000×1000, 4 потока

На рисунке 3 — запуск с 8 потоками. Достигнато наибольшее ускорение в серии; Results Match: True подтверждает корректность распараллеливания по строкам.

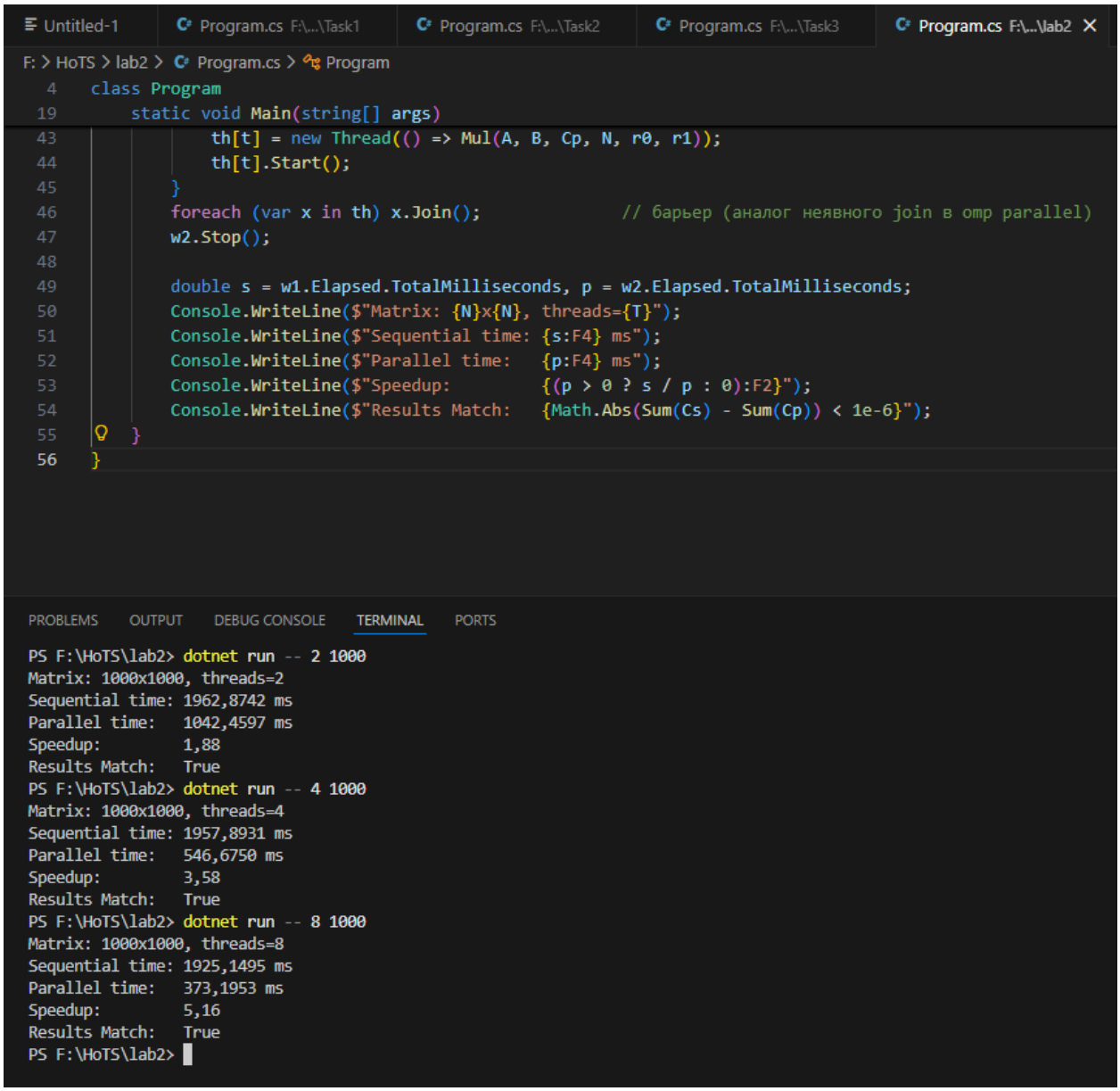


Рисунок 3 — Задача 4: перемножение матриц 1000×1000, 8 потоков

Сводка измерений приведена в таблице 1. Последовательное время совпадает во всех запусках благодаря фиксированному начальному зерну генератора (seed = 12345); ускорение монотонно растёт с числом потоков, но остаётся сублинейным.

Число потоков	Последовательное время, мс	Параллельное время, мс	Ускорение
2	1962,8742 мс	1042,4597 мс	1,88
4	1957,8931 мс	546,6750 мс	3,58
8	1925,1495 мс	373,1953 мс	5,16

Вопросы:

1. Почему в задаче не требуются мьютексы (критические секции)?

Потому что разделяемая запись отсутствует по построению. Каждый поток записывает только в принадлежащий ему непрерывный блок строк матрицы C, диапазоны не пересекаются; матрицы A и B после инициализации не изменяются и доступны всем потокам только на чтение. Чтение без изменения не порождает гонок, поэтому синхронизация в вычислительном ядре не нужна. Контрольная сумма считается главным потоком уже после барьера Join(), когда все рабочие потоки завершены, — и здесь конфликтов нет. Побочное следствие: параллельная версия даёт тот же результат, что и последовательная (подтверждается Results Match: True).

2. Как выбран порядок обхода матриц и зачем?

Выбран порядок i → k → j при одномерном хранении m[i*N+j]. При таком порядке для фиксированных i и k внутренний цикл по j читает строку B[k] последовательно по памяти, а множитель A[i*N+k] загружается один раз в регистр aik и переиспользуется на всейinner-итерации. Это максимизирует пространственную локальность и минимизирует промахи кэша. Классический порядок i → j → k при том же хранении заставлял бы читать B по столбцам с шагом N, что разрушает локальность и заметно замедляет обе версии.

3. Почему ускорение меньше линейного (меньше числа потоков)

Три причины. (1) Закон Амдала: часть работы выполняется последовательно — генерация матриц, аллокация, подсчёт контрольных сумм, создание и ожидание потоков; при p параллельной доле предельное ускорение $S \le 1/((1-p) + p/T)$. (2) Накладные расходы на параллелизм: порождение T

потоков и барьер Join() стоят времени, которое при малом N сопоставимо с полезной работой. (3) Ограничение пропускной способности памяти: с ростом потоков растёт число одновременных обращений к общим A и B, и система упирается в шину/кэш-иерархию, а не в число ядер. Поэтому кривая ускорения выпуклая и выходит на плато.

Вывод: Реализованы последовательная и параллельная версии перемножения матриц 1000×1000 на C# с распараллеливанием по строкам (аналог parallel for schedule(static)); барьер обеспечен Join().

Подтверждено, что при непересекающихся областях записи синхронизация не требуется, а параллельная версия воспроизводит последовательную точно (Results Match: True во всех запусках).

По таблице 1 установлено, что ускорение растёт с числом потоков, но остаётся сублинейным вследствие закона Амдала, накладных расходов на параллелизм и ограничения пропускной способности памяти; выбор порядка циклов i → k → j и одномерного хранения обоснован локальностью данных.